

Modelos de Interfaces

Dentro del contexto de ESL, los modelos de interfaces nos permiten comunicar diversos modelos entre sí. A través de TLM 2.0, podemos modelar interfaces a alto nivel sin entrar en aspectos protocolarios ni de la capa física.

Como vimos anteriormente, en alto nivel, nos concentramos en las transacciones como tal:

- Write
- Read

No nos concentramos en las señales:

- Valid
- Ready
- Done
- Data
- Address

Para ello, tenemos que saber que también tenemos el mismo nivel de abstracción en los modelos, pero para las interfaces pasamos de transacciones a algo arquitectónico y luego al RTL.

Modelo Transaccional en TLM 2.0

TLM 2.0 extiende SystemC para agregar la parte de transacciones. Podemos abstraer la comunicación entre un CPU y una memoria por transacciones de lectura y escritura, sin necesidad de tomar en cuenta señales, acelerando el proceso de desarrollo y simulación.

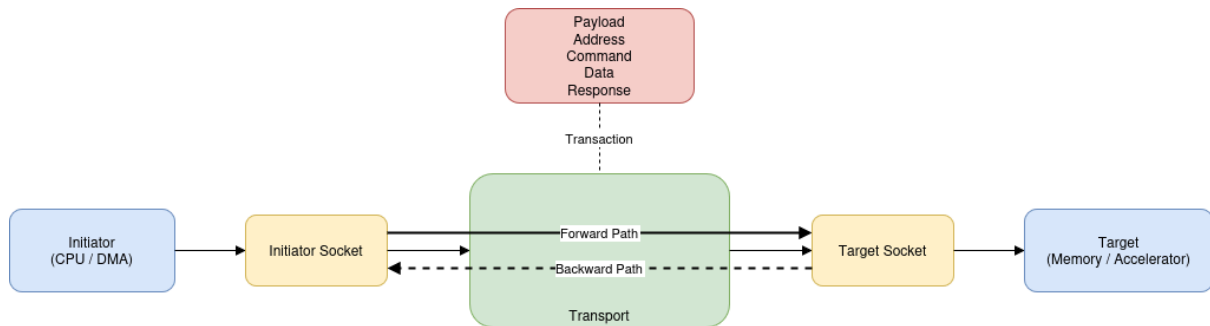


Figura 1. Arquitectura de TLM 2.0. Imagen generada con IA para efectos ilustrativos

Ya hemos visto los conceptos de initiators, target, payload y transport. Vamos a verlos más en detalle en esta sección:

Initiators

Inician las transacciones. Dentro de un sistema computacional, hay distintos iniciadores, tales como CPUs, DMAs, Controladores y Masters AXI. En este caso, los initiators son elementos que demandan accesos a datos.

Targets

Reciben y responden a las transacciones. Dentro de un sistema, podemos tener memorias, periféricos, aceleradores y clientes AXI.

Es importante destacar que existen dispositivos que pueden incluir ambas funcionalidades.

Payloads

Son objetos que contienen toda la información de la transacción. TLM 2.0 proporciona el `tlm::tlm_generic_payload`, que contiene:

- Dirección
- Datos
- Comandos
- Longitud
- Respuesta

Todos estos componentes se acceden mediante *accessors* set/get. Por ejemplo, para crear un mínimo, se puede usar

```
payload.set_command(TLM_READ_COMMAND);  
payload.set_address(0x1000);  
payload.set_data_ptr(buffer);
```

Los comandos disponibles son:

Enumerator	
<i>TLM_READ_COMMAND</i>	
<i>TLM_WRITE_COMMAND</i>	
<i>TLM_IGNORE_COMMAND</i>	

Mientras que los datos son buffers de datos crudos, que requieren de setear el *length* para poder determinar la longitud de los datos de la transacción. Ahora bien, el *address* es la dirección dentro del modelo del dispositivo que tendrá que actuar sobre los datos. Por ejemplo: una memoria que guarda los datos en una celda.

Para las respuestas, estas son puestas por los targets:

Enumerator	
<i>TLM_OK_RESPONSE</i>	
<i>TLM_INCOMPLETE_RESPONSE</i>	
<i>TLM_GENERIC_ERROR_RESPONSE</i>	
<i>TLM_ADDRESS_ERROR_RESPONSE</i>	
<i>TLM_COMMAND_ERROR_RESPONSE</i>	
<i>TLM_BURST_ERROR_RESPONSE</i>	
<i>TLM_BYTE_ENABLE_ERROR_RESPONSE</i>	

En teoría, el payload se devuelve al iniciador mediante una respuesta. Si es un comando de lectura hacia el target, el target setea los datos de interés y pone una respuesta. Si es una escritura, los datos no son necesarios.

Transport

El transport es el canal mediante el cual viajan los *payloads*. Tenemos varios tipos de transports:

- Blocking transport: *b_transport*. Detiene la ejecución hasta que el target responde. Esto es equivalente al que iniciador espera la respuesta. Una forma de abstraer un transport end de memoria:

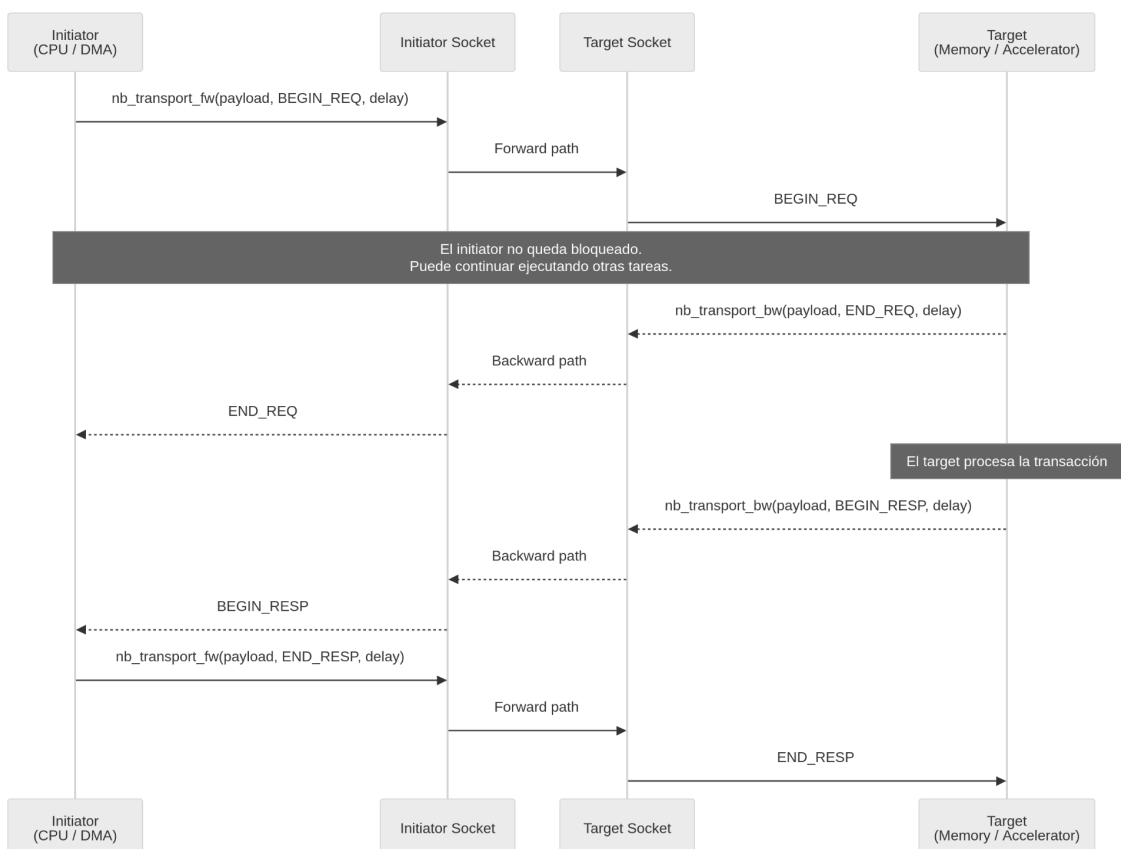
```

void memory::b_transport(
    tlm_generic_payload& trans,
    sc_time& delay)
{
    delay += sc_time(50, SC_NS);
}

```

Podemos tener la abstracción del tiempo. Sin embargo, en la práctica, el blocking transport puede ser muy abstracto e irrealista con muchos sistemas digitales actuales.

- Non-blocking transport: nb_transport_fw y nb_transport_bw. No realiza esperas durante las transacciones y puede enviar varias transacciones, que después serán respondidas. El _fw es donde el iniciador envía los payloads, mientras que el _bw es desde el target hasta el iniciador



Note que es un protocolo estilo handshaking. El target hace reconocimiento del request, mientras que el iniciador hace reconocimiento de la respuesta del target.

Ahora, desde la perspectiva del iniciator, éste envía los datos a través del forward path y recibe de vuelta en el backward path. En el target, la perspectiva es al revés.

Instalación del CPU:

```
void send_transaction(tlm::tlm_command cmd, std::uint64_t addr, std::uint32_t& value)
{
    tlm::tlm_generic_payload trans;
    tlm::tlm_phase phase = tlm::BEGIN_REQ;
    sc_core::sc_time delay = SC_ZERO_TIME;

    trans.set_command(cmd);
    trans.set_address(addr);
    trans.set_data_ptr(reinterpret_cast<unsigned char*>(&value));
    trans.set_data_length(sizeof(value));
    trans.set_streaming_width(sizeof(value));
    trans.set_byte_enable_ptr(nullptr);
    trans.set_dmi_allowed(false);
    trans.set_response_status(tlm::TLM_INCOMPLETE_RESPONSE);

    const auto status = socket->nb_transport_fw(trans, phase, delay);
    ...
}
```

```
SC_CTOR(CPU) : socket("socket")
{
    socket.register_nb_transport_bw(this, &CPU::nb_transport_bw);
    SC_THREAD(run);
}
```

Instalación en memoria:

```
SC_CTOR(Memory) : socket("socket")
{
    socket.register_nb_transport_fw(this, &Memory::nb_transport_fw);
    SC_THREAD(process_requests);
}
```

```

void process_requests()
{
    while (true)
    {
        wait(request_event);

        auto* trans = pending_trans;
        pending_trans = nullptr;

        if (trans == nullptr)
        {
            continue;
        }

        const auto cmd = trans->get_command();
        const auto addr = trans->get_address();
        auto* ptr = trans->get_data_ptr();
        const auto len = trans->get_data_length();

        if ((addr + len) > kSize)
        {
            trans->set_response_status(tlm::TLM_ADDRESS_ERROR_RESPONSE);
        }
        else if (cmd == tlm::TLM_WRITE_COMMAND)
        {
            std::memcpy(&data[addr], ptr, len);
            trans->set_response_status(tlm::TLM_OK_RESPONSE);
        }
        else if (cmd == tlm::TLM_READ_COMMAND)
        {
            std::memcpy(ptr, &data[addr], len);
            trans->set_response_status(tlm::TLM_OK_RESPONSE);
        }
        else
        {
            trans->set_response_status(tlm::TLM_COMMAND_ERROR_RESPONSE);
        }

        wait(sc_core::sc_time(10, sc_core::SC_NS));

        tlm::tlm_phase phase = tlm::BEGIN_RESP;
        sc_core::sc_time delay = sc_core::SC_ZERO_TIME;
        const auto status = socket->nb_transport_bw(*trans, phase, delay);

        if (status != tlm::TLM_COMPLETED && status != tlm::TLM_ACCEPTED)
        {
            SC_REPORT_WARNING("TLM-2", "Unexpected return from nb_transport_bw");
        }
    }
}
};

```

Representación de Interconexiones

Para interconexiones más complejas, se tiene que hacer un módulo que abstraiga la estructura de interconexión deseada. En otras palabras, existirá un módulo enrutador que permita conectar los distintos módulos, con arbitraje y protecciones.

Considerando un bus:

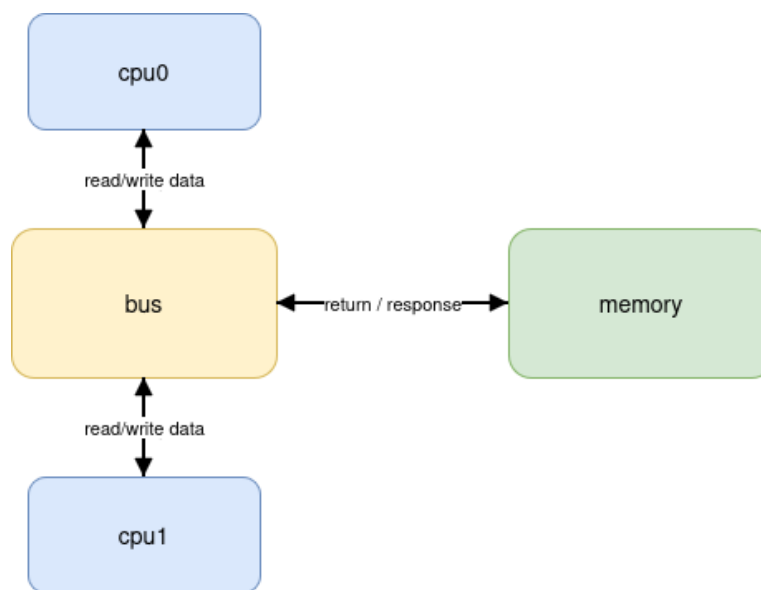


Figura 2. Diagrama de bloques de un bus en TLM. Imagen generada por IA.

Para intercomunicar dos procesadores con memoria, se debe crear un mecanismo de arbitraje que le permita a un CPU acceder a la memoria de forma ordenada y controlada. Si ambos procesadores logran acceder a la memoria, se generarán comportamientos no esperados (UB).

Ahora, con respecto a cómo se comporta el sistema, se presenta el diagrama de secuencias, donde se explica que se ocupa controlar el bus mediante un mutex, que se libera cuando se termina cada transacción.

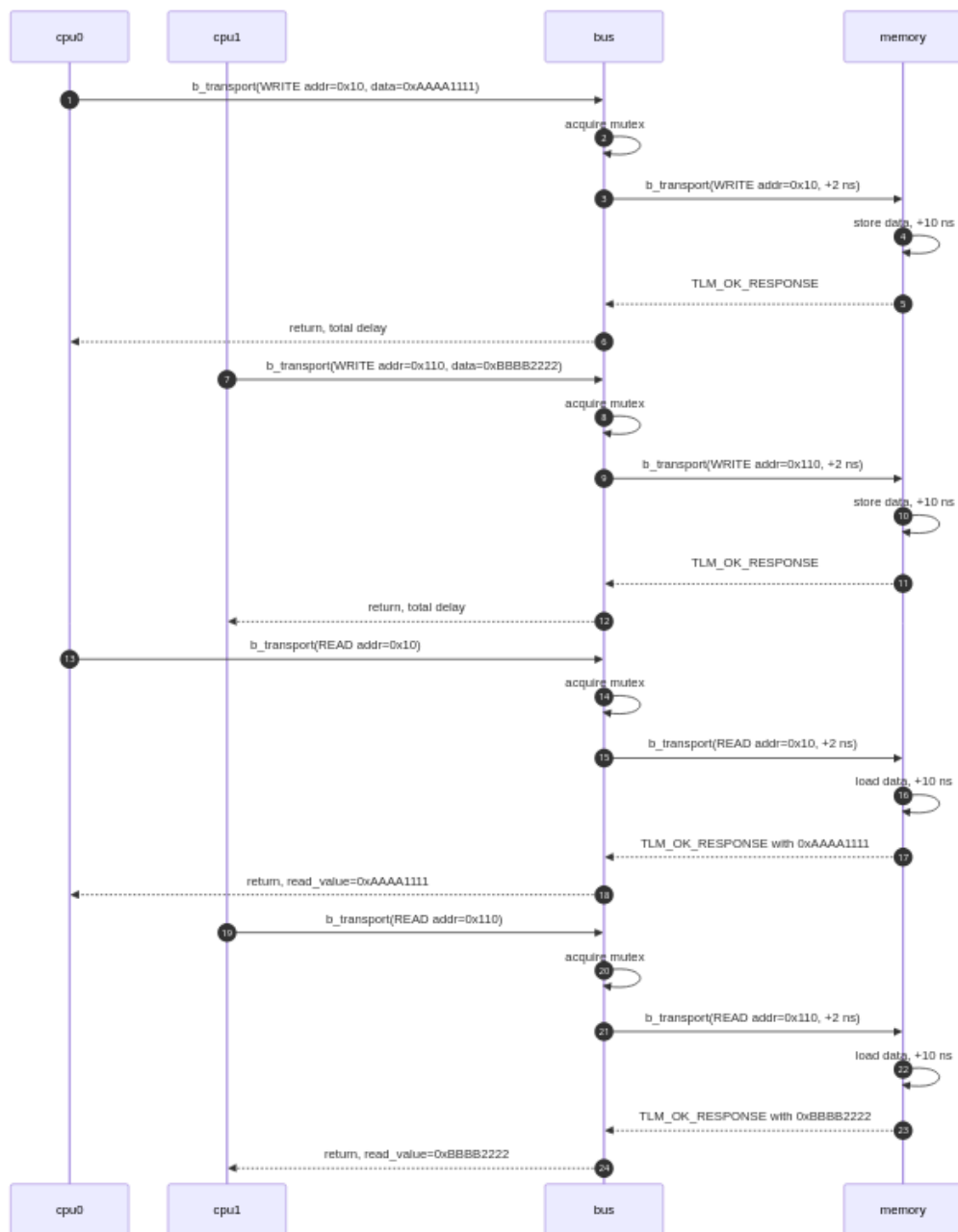


Figura 2. Diagrama de secuencias de un bus en TLM. Imagen generada por IA.

Lo mismo va a aplicar para otros mecanismos de interconexión.

Modelos Arquitectónicos de Interfaces

En este caso, vamos a considerar lo más cercano a las señales del protocolo AXI, que es uno de los más relevantes hoy en la industria.

Para entender las conexiones AXI4, estas especifican:

- Canal de direcciones
- Canal de datos
- Señales de reconocimiento en cada etapa:

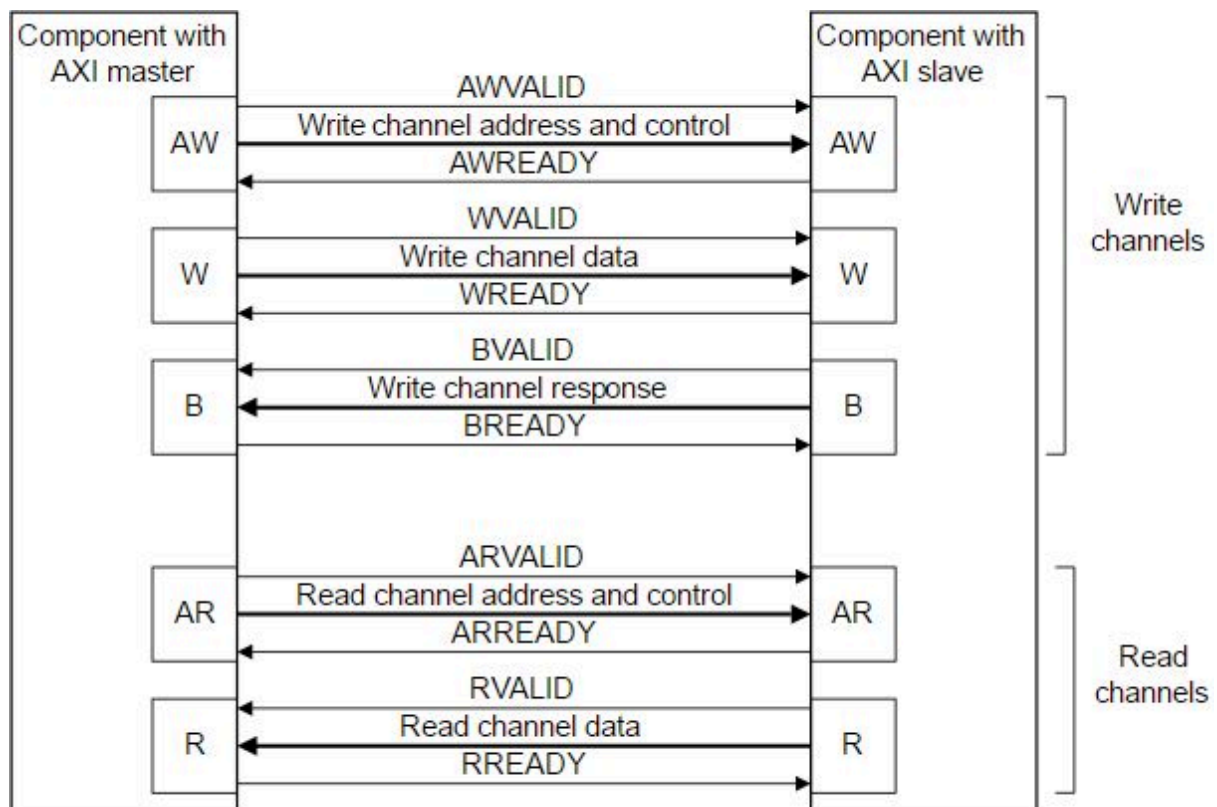


Figura 3. Diagrama del protocolo AXI4. Imagen obtenida en la [web](#).

Las transacciones se definen o se comportan muy similar a como vimos en TLM. Sin embargo, estamos manejando un nivel de implementación o microarquitectura, por lo que llegamos a entenderlo a nivel de secuencias:

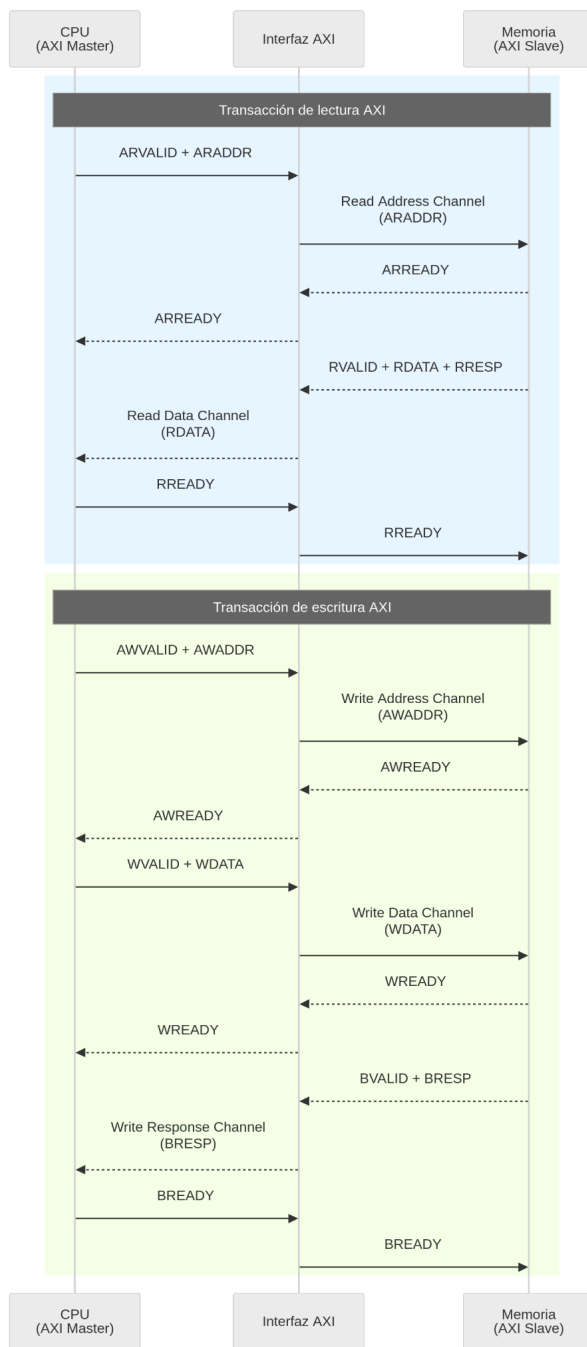


Figura 2. Diagrama de secuencias del protocolo AXI4. Imagen generada por IA.

A nivel de RTL, debería comportarse de la siguiente manera:

1.3.3 Write burst example

Figure 1-6 shows a write transaction. The process starts when the master sends an address and control information on the write address channel. The master then sends each item of write data over the write data channel. When the master sends the last data item, the **WLAST** signal goes HIGH. When the slave has accepted all the data items, it drives a write response back to the master to indicate that the write transaction is complete.

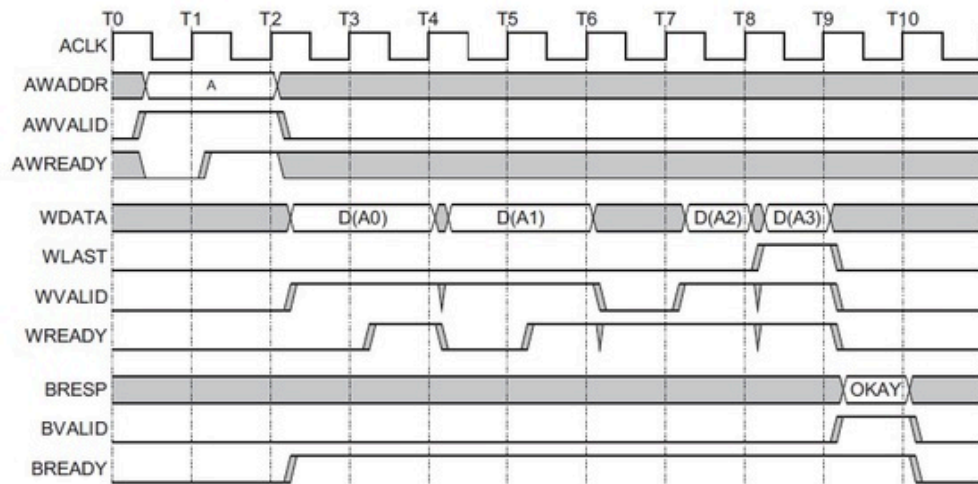


Figure 1-6 Write burst

Figura 3. Diagrama de señales del protocolo AXI4. Imagen obtenida en la [web](#).

Tareas

1. Preparación de exposición sobre patrones de diseño top-down y bottom-up (próxima semana)
2. Primer entrega del proyecto: Estudio Preliminar y Tema.

Referencias

TLM

2.0:

https://eda-playground.readthedocs.io/en/latest/_static/systemc-2.3.1/tlm/index.html

AXI4:

<https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html>